

Database Systems-Relational Database

Nitin Awathare

Department of Computer Science and Engineering,
IIT Jodhpur

1



Outline

- Structure of Relational Databases
- Database Schema
- Keys
- Schema Diagrams
- Relational Query Languages
- The Relational Algebra

Relational Database

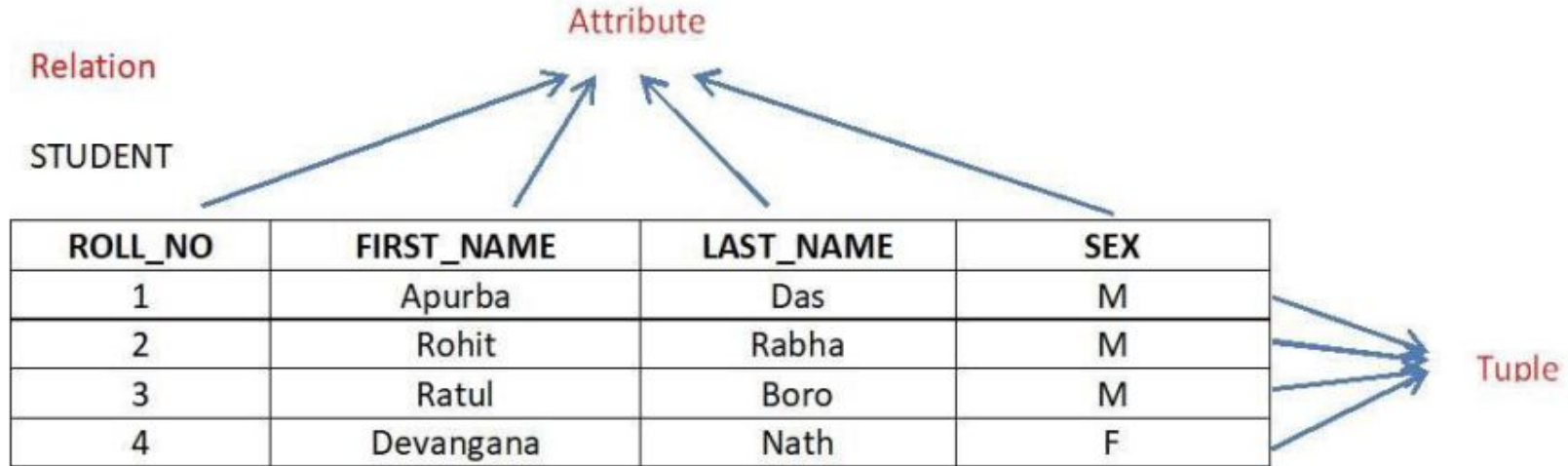
- Relational Database Management System (RDBMS)
 - Consists of a number of *tables* and single *schema* (definition of tables and attributes)
 - Students (sid, name, login, age, gpa)

Students identifies the table

sid, name, login, age, gpa identify attributes

sid is primary key

Example of Student Relation

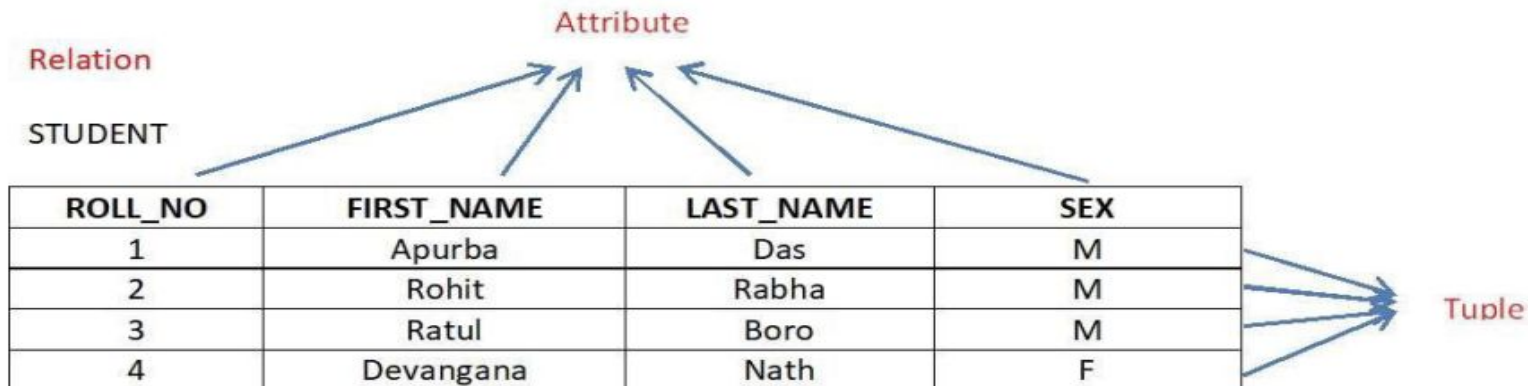


Attributes

- The set of allowed values for each attribute is called the **domain** of the attribute
- Attribute values are (normally) required to be **atomic**; that is, indivisible
- The special value ***null*** is a member of every domain. Indicated that the value is “unknown”
- The null value causes complications in the definition of many operations

Database Schema and Instances

- **Database schema** -- is the logical structure of the database.
- **Database instance** -- is a snapshot of the data in the database at a given instant in time.
- Example:
 - schema: student (*roll_no*, *first_name*, *last_name*, *sex*)
 - Instance:



Keys

- Let $K \subseteq R$
- K is a **superkey** of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$
 - Example: $\{ID\}$ and $\{ID, name\}$ are both superkeys of *instructor*.
- Superkey K is a **candidate key** if K is minimal
Example: $\{ID\}$ is a candidate key for *Instructor*
- One of the candidate keys is selected to be the **primary key**.
 - Which one?

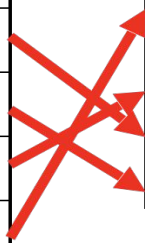
Keys

- **Foreign key** constraint: Value in one relation must appear in another
 - *dept_name* in *instructor* is a foreign key from *instructor* referencing *department*

Enrolled		Foreign key
<u>cid</u>	grade	studid
Carnatic101	C	53831
Reggae203	B	53832
Topology112	A	53650
History 105	B	53666

Student

<u>sid</u>	name	login
50000	Dave	dave@cs
53666	Jones	jones@cs
53688	Smith	smith@ee
53650	Smith	smith@math
53831	Madayan	madayan@music
53832	Guldu	guldu@music



Relational Query Languages

- Procedural versus non-procedural, or declarative
- “Pure” languages:
 - Relational algebra
 - Tuple relational calculus
 - Domain relational calculus
- The above 3 pure languages are equivalent in computing power
- We will concentrate in this chapter on relational algebra

Relational Algebra

- Collection of operators for specifying queries
- Query describes step-by-step procedure for computing answer (i.e., *operational*)
- Each operator accepts one or two relations as input and returns a relation as output
- Relational algebra expression composed of multiple operators

Basic Operations

- select: σ
- project: Π
- union: $\cup$
- set difference: $-$
- Cartesian product: $\times$
- rename: ρ
- Other operators can be defined in terms of basic operators

Example Schema

- Courses (cid, instructor, quarter, dept)
- Students (sid, name, gpa)
- Enrolled (cid, grade, studid)

sid	name	gpa
50000	Dave	3.3
53666	Jones	3.4
53688	Smith	3.2
53650	Smith	3.8
53831	Madayan	1.8
53832	Guldu	2.0

Select Operation

- Select students with gpa higher than 3.3 from S1:
- $\sigma_{gpa>3.3}(S1)$

sid	name	gpa
50000	Dave	3.3
53666	Jones	3.4
53688	Smith	3.2
53650	Smith	3.8
53831	Madayan	1.8
53832	Guldu	2.0



sid	name	gpa
53666	Jones	3.4
53650	Smith	3.8

Project Operation

- Project name and gpa of all students in S1:
- $\Pi_{\text{name, gpa}}(S1)$

Sid	name	gpa
50000	Dave	3.3
53666	Jones	3.4
53688	Smith	3.2
53650	Smith	3.8
53831	Madayan	1.8
53832	Guldu	2.0



name	gpa
Dave	3.3
Jones	3.4
Smith	3.2
Smith	3.8
Madayan	1.8
Guldu	2.0

Combine Select and Project Operation

- Project name and gpa of students in S1 with gpa higher than 3.3: $\Pi_{\text{name, gpa}}(\sigma_{\text{gpa} > 3.3}(\text{S1}))$

Sid	name	gpa
50000	Dave	3.3
53666	Jones	3.4
53688	Smith	3.2
53650	Smith	3.8
53831	Madayan	1.8
53832	Guldu	2.0



name	gpa
Jones	3.4
Smith	3.8

Cartesian-Product Operation

- The Cartesian-product operation (denoted by $\times$) allows us to combine information from any two relations.
- Example: the Cartesian product of the relations *instructor* and *teaches* is written as:

instructor $\times$ *teaches*

- We construct a tuple of the result out of each possible pair of tuples: one from the *instructor* relation and one from the *teaches* relation (see next slide)
- Since the instructor *ID* appears in both relations we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came.
 - *instructor.ID*
 - *teaches.ID*

Join Operation

- Combine information from two or more tables
- Example: students enrolled in courses: S1

 S1.cid=E.studid

Sid	name	gpa
50000	Dave	3.3
53666	Jones	3.4
53688	Smith	3.2
53650	Smith	3.8
53831	Madayan	1.8
53832	Guldu	2.0

<u>cid</u>	grade	studid
Carnatic101	C	53831
Reggae203	B	53832
Topology112	A	53650
History 105	B	53666

Join Operation

S1

Sid	name	gpa
50000	Dave	3.3
53666	Jones	3.4
53688	Smith	3.2
53650	Smith	3.8
53831	Madayan	1.8
53832	Guldu	2.0

E

<u>cid</u>	grade	studid
Carnatic101	C	53831
Reggae203	B	53832
Topology112	A	53650
History 105	B	53666

sid	name	gpa	cid	grade	studid
53666	Jones	3.4	History105	B	53666
53650	Smith	3.8	Topology112	A	53650
53831	Madayan	1.8	Carnatic101	C	53831
53832	Guldu	2.0	Reggae203	B	53832

Set Operation

- Union ($R \cup S$)
 - All tuples in R or S (or both)
 - R and S must have same number of fields
 - Corresponding fields must have same domains
- Intersection ($R \cap S$)
 - All tuples in both R and S
- Set difference ($R - S$)
 - Tuples in R and not S
- Cross product or Cartesian product ($R \times S$)
 - All fields in R followed by all fields in S
 - One tuple (r,s) for each pair of tuples $r \in R, s \in S$

Set Operation

Example: Intersection

S1

sid	name	gpa
50000	Dave	3.3
53666	Jones	3.4
53688	Smith	3.2
53650	Smith	3.8
53831	Madayan	1.8
53832	Guldu	2.0

S2

sid	name	gpa
53666	Jones	3.4
53688	Smith	3.2
53700	Tom	3.5
53777	Jerry	2.8
53832	Guldu	2.0

$S1 \cap S2 =$

sid	name	gpa
53666	Jones	3.4
53688	Smith	3.2
53832	Guldu	2.0

The Assignment Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The assignment operation is denoted by $\leftarrow$ and works like assignment in a programming language.
- Example: Find all instructor in the “Physics” and Music department.

$$Physics \leftarrow \sigma_{dept_name="Physics"}(instructor)$$

$$Music \leftarrow \sigma_{dept_name="Music"}(instructor)$$

$$Physics \cup Music$$

- With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.

The Rename Operation

- The results of relational-algebra expressions do not have a name that we can use to refer to them. The rename operator, ρ , is provided for that purpose
- The expression:

$$\rho_x(E)$$

returns the result of expression E under the name x

- Another form of the rename operation:

$$\rho_{x(A1, A2, \dots, An)}(E)$$

Aggregation Operation

- These functions operate on the multiset (set with duplicates) of values of a column of a relation, and return a value

avg: average value

min: minimum value

max: maximum value

sum: sum of values

count: number of values

- Find the average salary of instructors
 - $\gamma_{avg(salary)}(instructor)$
- Find the total number of courses in 2018
 - $\gamma_{count(course_id)}(\sigma_{year=2018}(section))$

Equivalent Queries

- There is more than one way to write a query in relational algebra.
- Example: Find information about courses taught by instructors in the Physics department with salary greater than 90,000
- Query 1

$$\sigma_{dept_name="Physics" \wedge salary > 90,000} (instructor)$$

■ Query 2

$$\sigma_{dept_name="Physics"} (\sigma_{salary > 90,000} (instructor))$$

- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.

Summary

- Algebras are useful to manipulate data types (relations in this case)
- Set-oriented
- Brings some clarity to what needs to be done
- Opportunities for optimization
 - May have different expressions that do same thing
- We will see examples of algebras for other types of data in this course

THANK YOU